

Sri Sivasubramaniya Nadar College of Engineering

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

III Year CSE (V Semester) | Academic Year 2026-27

UCS3513 - SYSTEM DESIGN LABORATORY

Lab Exercise 2

Design and Implement a Basic URL Shortener Using Spring Boot

Submitted by,

Name	Register Number
Tushyent N P	3122 24 5001 189
Vino S R Harrison	3122 24 5001 199
Venkat Prabhu S	3122 24 5001 196
Vignesh Raaj S	3122 24 5001 198

Design and Implement a Basic URL Shortener Using Spring Boot

Aim

- To design and implement a basic URL Shortener application using Spring Boot that generates a short URL for a given long URL using a hashing technique.
- To implement URL redirection that maps short codes back to original URLs, with collision handling and persistent storage using MongoDB.

Question

Design and implement a basic URL Shortener using Spring Boot with the following requirements:

1. Accept a long URL from the user.
2. Generate a short code using SHA-256 hashing algorithm.
3. Use the first 6 characters of the generated hash as the short code.
4. Check the database to ensure the generated short code is unique. If it already exists for a different URL, append "A" until unique.
5. Store the mapping between the long URL and the short code in MongoDB.
6. Redirect the user to the original URL when the generated short URL is accessed via GET `/shortCode`.

Sample Input / Output

Sample Input (POST /shorten):

```
{  
  "longUrl": "https://www.google.com/search?q=system+design"  
}
```

Sample Output:

```
{  
  "shortUrl": "http://localhost:8080/aB12Xy"  
}
```

When the generated short URL is accessed in a browser, it redirects to the original long URL via HTTP 302 redirect.

System Architecture & Workflow

The following diagram illustrates the complete architecture of the URL Shortener system, showing the flow from client request through the Spring Boot REST controller, service layer, hashing utility, and MongoDB persistence.

Procedure (Workflow)

- 1) **Client sends a POST request** to the `/shorten` endpoint with a JSON body containing the `longUrl` field.
- 2) **The controller receives the request** and extracts the `longUrl`, forwarding it to the `UrlService`.
- 3) **The service checks the database** (`UrlRepository.findBylongUrl()`) to see if this URL has already been shortened.
- 4) **If found**, the existing short URL is returned immediately (idempotency).
- 5) **If not found**, `HashUtil.generateShortCode()` computes the SHA-256 hash of the URL and takes the first 6 hex characters.
- 6) **Collision check**: The service verifies uniqueness via `repo.existsByShortCode()`. If the short code already exists, it appends "A" repeatedly until unique.
- 7) **A new `UrlModel` document** is created with `longUrl`, `shortCode`, and `createdAt` timestamp, then saved to MongoDB.
- 8) **The service returns** the generated short URL (`http://localhost:8080/<shortCode>`) as a JSON response.
- 9) **For redirection**, the client accesses `GET /{shortCode}`. The controller calls `service.getOriginalUrl()` to look up the original URL.
- 10) **A 302 redirect response** is sent with the `Location` header set to the original URL, causing the browser to navigate there.

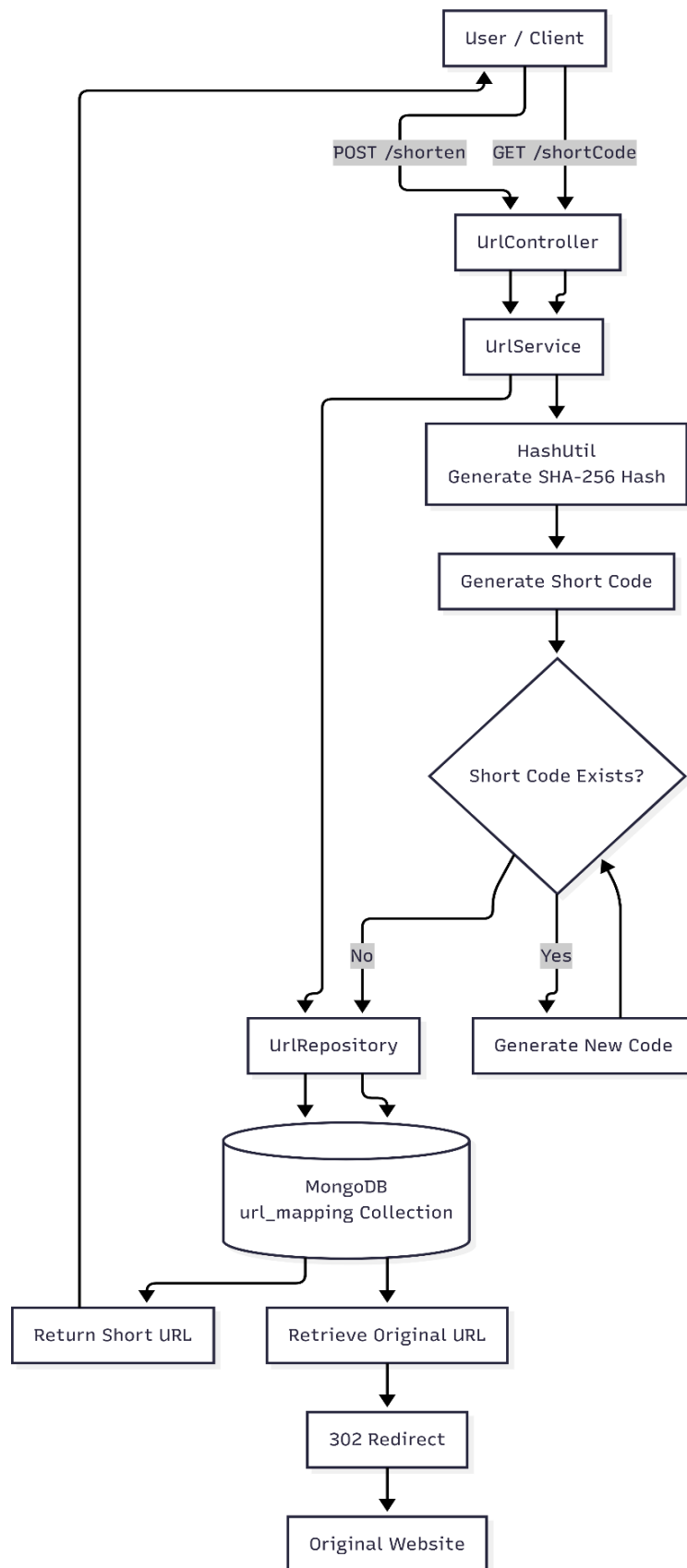


Figure 1: System Architecture Diagram — URL Shortener with Spring Boot and MongoDB

Design of URL Shortener

1. Hashing Technique (SHA-256)

The system uses SHA-256 (Secure Hash Algorithm 256-bit) via Java's `java.security.MessageDigest` class. The long URL is converted to bytes and hashed, producing a 64-character hexadecimal string. Only the first 6 characters are used as the short code, giving a key space of $16^6 = 16,777,216$ possible short codes.

```
public static String generateShortCode(String longUrl) {  
    MessageDigest md = MessageDigest.getInstance("SHA-256");  
    byte[] hash = md.digest(longUrl.getBytes());  
    StringBuilder sb = new StringBuilder();  
    for (byte b : hash) {  
        sb.append(String.format("%02x", b));  
    }  
    return sb.substring(0, 6); // First 6 hex chars  
}
```

2. Collision Handling Strategy

Collisions occur when two different long URLs produce the same 6-character hash prefix. The system handles this by:

- Checking the database using `repo.existsByShortCode(shortCode)` before saving.
- If the short code already exists (collision detected), the character "A" is appended to the short code (e.g., "a1b2c3" → "a1b2c3A").
- The check runs in a while-loop until a unique short code is found.
- This approach is simple but guarantees uniqueness without requiring re-hashing.

3. Uniqueness Verification

Two levels of uniqueness are maintained:

- Long URL uniqueness: Before generating a new short code, the system checks if the long URL was already shortened (query by `longUrl`). If found, the existing short URL is returned — preventing duplicate entries.
- Short code uniqueness: Before saving, the system verifies that the short code does not already exist in the database. If it does, the code is extended until unique.

4. URL Mapping Mechanism

The mapping between long URLs and short codes follows this flow:

- **Shortening (POST /shorten):**

Long URL → SHA-256 hash → 6-char prefix → collision check → save to MongoDB → return short URL

- **Redirect (GET /{shortCode}):**

Short code → query MongoDB → retrieve long URL → HTTP 302 redirect

5. Data Model (MongoDB Document)

Field	Type	Description
_id	String (ObjectId)	Auto-generated MongoDB document identifier
longUrl	String	The original long URL submitted by the user
shortCode	String	The generated unique short code (hash prefix)
createdAt	String	Timestamp of when the mapping was created (ISO format)

6. Example Document in MongoDB

```
{  
  "_id": "689df8d9f4c3a5b9c4e7123",  
  "longUrl": "https://www.google.com/search?q=system+design",  
  "shortCode": "aB12Xy",  
  "createdAt": "2026-07-09T13:45:30"  
}
```

Output Screenshots

The following screenshots demonstrate the application being tested using Postman and a web browser.

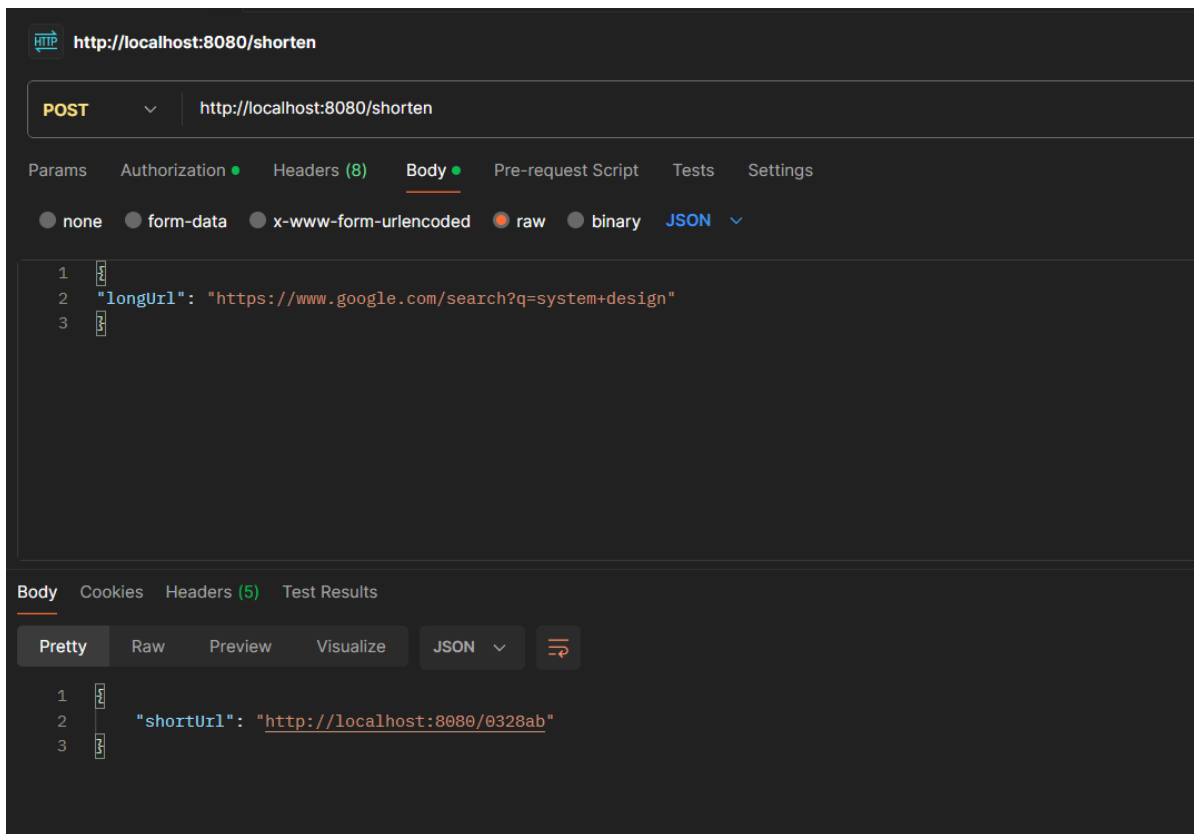


Figure 2: Postman — POST /shorten request and response showing generated short URL

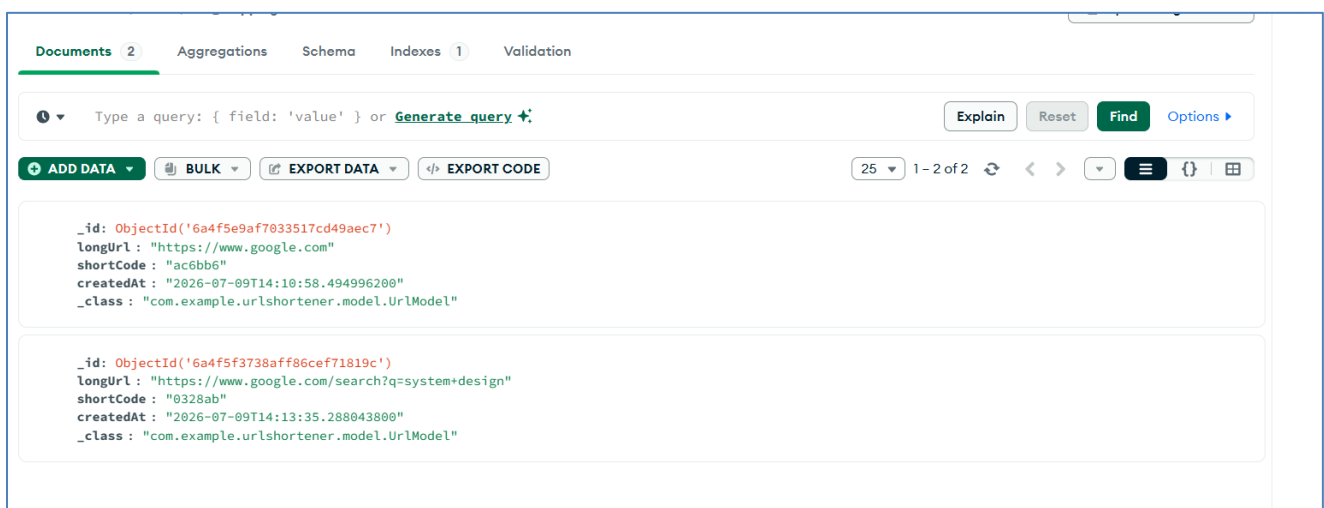


Figure 3: Browser — Redirect from short URL to original Google search page

API Endpoints & Test Results

Table 1: REST API Endpoints

#	Method	Endpoint	Description	Status Code
1	POST	/shorten	Create shortened URL	200 OK
2	GET	/{"shortCode"}	Redirect to original URL	302 Found
3	GET	/{"shortCode"}	Invalid short code	500 Internal Error

Table 2: Technology Stack

#	Technology / Tool	Purpose
1	Java 25	Programming language
2	Spring Boot 4.1.0	Framework for REST API development
3	Spring Data MongoDB	Database access and document mapping
4	MongoDB	NoSQL database for URL mappings
5	SHA-256	Hashing algorithm for short code generation
6	Postman	API testing tool

Table 3: Test Case Results

#	Test Case	Input	Expected Result	Status
1	Shorten new URL	{"longUrl":"https://example.com"}	Short URL returned	✓ Pass
2	Shorten same URL again	{"longUrl":"https://example.com"}	Same short URL (idempotent)	✓ Pass
3	Redirect valid short code	GET /a1b2c3	302 → original URL	✓ Pass
4	Redirect invalid code	GET /invalid	500 / error	✓ Pass

Learning Outcomes

After completing this experiment, the team has gained the following competencies:

- Understood the architecture and workflow of a URL Shortener system using Spring Boot and MongoDB.
- Developed RESTful APIs (POST /shorten and GET /{shortCode}) to generate short URLs and redirect users to original URLs.
- Implemented SHA-256 hashing to generate short codes and handled short-code collisions by appending characters until uniqueness.
- Used Spring Data MongoDB to store and retrieve URL mappings efficiently in a NoSQL database.
- Gained hands-on experience with Spring Boot framework, dependency injection, layered architecture (Controller → Service → Repository), and REST API testing using Postman.
- Learned about HTTP redirect mechanisms (302 status code) and idempotent URL shortening to prevent duplicate database entries.